

Prime Attention for Multivariate Time Series Forecasting (Simplified Reproduction)

Model Research Report (Sample) — based on Lee & Clark, *Dynamic Relational Priming Improves Transformer in Multivariate Time Series* (arXiv:2509.12196)

1. Overview

This report describes a simplified reproduction of 'prime attention'. Standard attention presents the same static token representation in every pair-wise interaction. Prime attention instead modulates each key/value vector with a pair-specific 'primer' vector F_{ij} , so that different channel pairs (lead-lag pairs vs. instantaneously correlated pairs vs. independent pairs) interact through tailored representations, at the same asymptotic cost as standard attention. The paper reports up to 6.5% forecasting accuracy gains on MTS benchmarks. Task here: multivariate forecasting on synthetic data with heterogeneous inter-channel relationships. The implementation must run in pure NumPy/SciPy/scikit-learn (no PyTorch or TensorFlow).

2. Model Formulation

Data: $C = 8$ channels, $T = 2000$ timesteps, generated with seed 42. Channel 0 = sine wave + AR(1) driver. Channel 1 = channel 0 delayed by 3 steps plus small noise (lead-lag pair). Channel 2 = $0.8 \times$ channel 0 plus noise (instantaneous correlation). Channel 3 = independent AR(1). Channels 4 and 5 = another lead-lag pair with lag 5 and different AR dynamics. Channels 6 and 7 = independent seasonal series plus noise. Build sliding windows with look-back $L_{in} = 24$ and horizon $H = 4$. For each window, embed each channel's look-back vector into a token z_c in R^d via a fixed seeded Gaussian random projection, with $d_{model} = 16$.

Standard attention: $q_i = z_i W_q$, $k_j = z_j W_k$, $v_j = z_j W_v$ with fixed seeded random projection matrices W in $R^{(d \times d)}$; weights $a_{ij} = \text{softmax}_j(q_i \cdot k_j / \sqrt{d})$; output $o_i = \sum_j a_{ij} v_j$.

Prime attention: compute lead-lag cross-correlation features R_{ij} in R^8 between the look-back series of channels i and j via FFT circular cross-correlation $R = \text{IFFT}(\text{FFT}(x_j) * \text{conj}(\text{FFT}(x_i)))$, keeping the first 8 lags and applying $\tanh$. The primer is $F_{ij} = 1 + \tanh(R_{ij} @ W_p)$, element-wise in R^d , where W_p in $R^{(8 \times d)}$ is the only learned matrix (initialized near zero so the primer starts as the identity modulation). Modulated key/value: $k_{\text{tilde}_j} = k_j * F_{ij}$ and $v_{\text{tilde}_j} = v_j * F_{ij}$ (element-wise). Output $o_{\text{tilde}_i} = \sum_j \text{softmax}_j(q_i \cdot k_{\text{tilde}_j} / \sqrt{d}) v_{\text{tilde}_j}$.

Readout: predict the next H values of channel i with ridge regression on the attention output only: o_i for the standard variant or o_{tilde_i} for the prime variant (do NOT concatenate z_i , so the attention layer is the information bottleneck).

3. Hyper-parameters

$d_{model} = 16$; number of lag features = 8; ridge alpha = 1.0; look-back 24; horizon 4; all random projections seeded with 42. Since no autodiff framework is available, fit W_p by evaluating 20 seeded random candidate matrices (scaled to small norm) plus the zero matrix and keeping the one with the lowest validation MSE. Target runtime under 2 minutes on CPU.

4. Training and Evaluation

Split windows chronologically 70% train / 15% validation / 15% test. Fit the ridge readout on train; select W_p on validation. Report average test MSE and MAE over channels and horizons for three models: (a) channel-independent ridge baseline (no attention, ridge on z_i only), (b) standard attention, (c) prime attention. Print a results table, the relative improvement of prime over standard attention, and a per-channel breakdown. Expect prime attention to beat standard attention (around 1% lower MSE on this data), with the clearest gains on the lead-lag channels 1 and 5.

5. Implementation Notes

Implement in Python with numpy, scipy, scikit-learn and pandas only. Provide a class PrimeAttentionForecaster with fit(X) and predict(X) methods plus a runnable main.py demo script: if no CSV file is provided, generate the synthetic 8-channel dataset described above with seed 42, run the full evaluation, and print the results table.

6. Reference

Hunjae Lee, Corey Clark. Dynamic Relational Priming Improves Transformer in Multivariate Time Series. arXiv:2509.12196, 2025. Official code: <https://github.com/timlee0131/Prime-Attention>